

DOCUMENTACIÓN MAPAS PROYECTO

Índice

Método utilizado.....	3
Instalación.....	3
Cómo empezar.....	3
Modelos.....	3
SellerProfile.....	4
PickupPoint.....	5
Controlador – MapController.....	6
Bloque 1.....	7
Bloque 2.....	7
Bloque 3.....	7
Vista.....	8
Archivo web.php.....	9
Resultado en el navegador.....	9
Añadir un marcador de inicio.....	11
Personalizar el mapa.....	12

Método utilizado

Hago uso del paquete *larswiegers/laravel-maps*, que soporta **Leaflet** y permite usar mapas gratuitos de OpenStreetMap (**desarrollar más esta explicación**)

Instalación

Desde visual studio, desde una terminal abierta dentro del proyecto, luego de haber levantado los contenedores, se introduce el comando **./vendor/bin/sail composer require larswiegers/laravel-maps** para la instalación del paquete.

Cómo empezar

Para empezar, quiero mostrar un mapa de prueba con puntos de recogida de ejemplo para comprobar el funcionamiento básico del mapa.

Modelos

Quiero mostrar en los puntos de recogida, el nombre de la empresa de cada vendedor, por lo que necesito un modelo para el perfil de los vendedores (**seller_profiles**) y un modelo para los puntos de recogida (**pickup_points**), y relacionar estos modelos entre ellos (un vendedor puede tener varios puntos de recogida pero un punto de recogida pertenece a un vendedor) para así poder hacer consultas SQL más sencillas y rápidas.

SellerProfile

```
app > Models > SellerProfile.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class SellerProfile extends Model
8  {
9      protected $table = 'seller_profiles';
10     protected $primaryKey = 'seller_id';
11
12     protected $fillable = [
13         'store_name',
14         'description',
15         'nif',
16         'banner_url'
17     ];
18
19     protected $hidden = [
20         'created_at',
21         'updated_at'
22     ];
23
24 }
25
```

Modelo SellerProfile

El modelo del perfil de los vendedores es básico, solamente tiene los datos sobre la **tabla**, la **clave primaria** (seller_id) y aquellos datos que se pueden rellenar mediante un formulario, y los que están ocultos. Para relacionar el perfil de los vendedores con los respectivos puntos de recogida, lo haré en el modelo de estos últimos.

PickupPoint

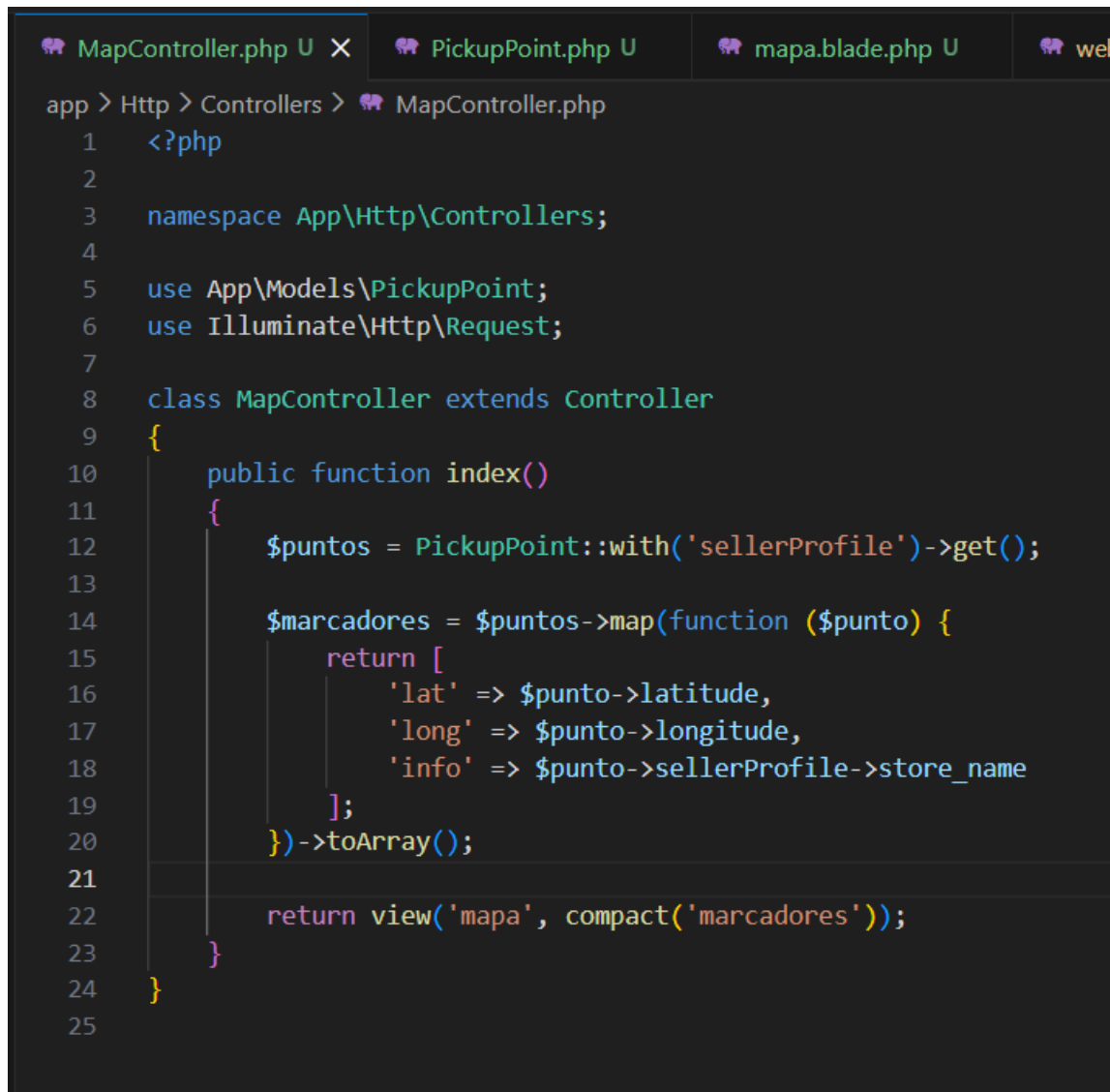
```
app > Models > PickupPoint.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class PickupPoint extends Model
8  {
9      protected $table = 'pickup_points';
10
11     protected $fillable = [
12         'latitude',
13         'longitude',
14         'address'
15     ];
16
17     protected $hidden = [
18         'seller_id',
19         'created_at',
20         'updated_at'
21     ];
22
23     public function sellerProfile(){
24         return $this->belongsTo(SellerProfile::class, 'seller_id');
25     }
26 }
27
```

Modelo PickupPoint

El modelo de los puntos de recogida, a parte de los datos que también hay en todos los modelos, también incluye una función, ya que un modelo es también una clase, por lo que puede tener funciones como todas las clases. Esta función es la que permite relacionar cada objeto de punto de recogida con cada perfil de vendedor mediante el id (**seller_id**).

Controlador – MapController

Luego de tener los modelos listos, ahora es necesario un controlador para poder acceder a los datos de la base de datos y poder obtener una lista de todos los puntos de recogida asociados a cada vendedor y mostrarlos en el mapa.



```
MapController.php X PickupPoint.php U mapa.blade.php U wel
app > Http > Controllers > MapController.php
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\PickupPoint;
6  use Illuminate\Http\Request;
7
8  class MapController extends Controller
9  {
10     public function index()
11     {
12         $puntos = PickupPoint::with('sellerProfile')->get();
13
14         $marcadores = $puntos->map(function ($punto) {
15             return [
16                 'lat' => $punto->latitude,
17                 'long' => $punto->longitude,
18                 'info' => $punto->sellerProfile->store_name
19             ];
20         }->toArray());
21
22         return view('mapa', compact('marcadores'));
23     }
24 }
25
```

Controlador del mapa (MapController)

El controlador está compuesto de varios bloques:

Bloque 1

Aquí, se define el array de los puntos de recogida (**puntos**), que contiene además de los datos de cada punto de recogida, un array asociado a cada punto con los datos del respectivo vendedor. Es como si se hiciera un *SELECT ** de los puntos de recogida más un *JOIN* para obtener los datos de los vendedores.

Bloque 2

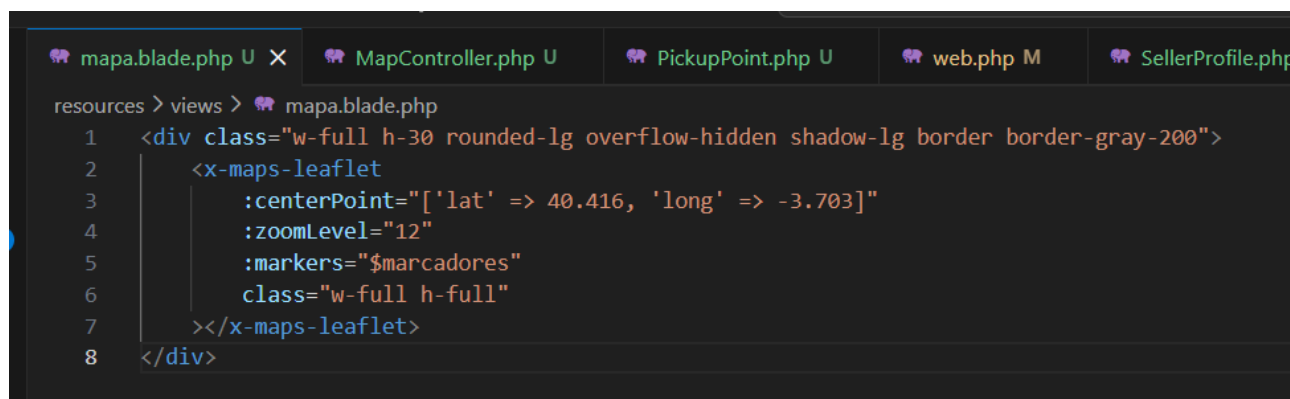
En este bloque, lo que se pretende es obtener las coordenadas de cada punto de recogida, más el nombre de la empresa del respectivo vendedor de cada punto, eliminando así, la información sobrante que no es útil para el mapa.

El array se almacena en la variable **marcadores**, y con la función **map**, se recorre todo el array de puntos obtenido en el bloque 1, y cada punto obtenido, se pasa a una función dentro de la función map, donde temporalmente se le llama **punto** a cada objeto del array, se **devuelven** las **coordenadas** del punto más la **info** del **vendedor** (el nombre de la empresa), y a medida que se obtienen los datos de cada punto, se guardan en un array con la función **toArray** (esto se hace porque la función map devuelve una colección, que es un objeto de Laravel que envuelve a un array para poder aplicar funciones sobre él, y el mapa de Leaflet no acepta colecciones para mostrar los marcadores, se necesitan arrays).

Bloque 3

Finalmente, una vez se obtiene el array con los datos principales de los puntos de recogida, se pasa este array a la vista donde está el mapa para así poder mostrar los marcadores.

Vista

A screenshot of a code editor with a dark theme. At the top, there are five tabs: 'mapa.blade.php U' (active), 'MapController.php U', 'PickupPoint.php U', 'web.php M', and 'SellerProfile.php'. The main editor area shows the content of 'mapa.blade.php' with the following code:

```
resources > views > mapa.blade.php
1 <div class="w-full h-30 rounded-lg overflow-hidden shadow-lg border border-gray-200">
2   <x-maps-leaflet
3     :centerPoint="['lat' => 40.416, 'long' => -3.703]"
4     :zoomLevel="12"
5     :markers="$marcadores"
6     class="w-full h-full"
7   ></x-maps-leaflet>
8 </div>
```

Código de la vista que muestra el mapa

Este es un código básico utilizado para mostrar un mapa de prueba con marcadores de ejemplo. Se emplea **Tailwind CSS**, que es una sintaxis CSS de Laravel que permite escribir CSS dentro del mismo archivo HTML para ahorrar trabajo. Aunque de normal, se sigue la norma de separar HTML, CSS y JavaScript en diferentes archivos, en frameworks como Laravel, se prefiere combinar CSS dentro del HTML para ahorrar problemas como sería borrar un código CSS que ya no utilizas pero que se te olvida quitarlo también en el HTML, etc

Además, también trae ciertas reglas de medida entre otras cosas que ahorran tiempo a la hora de decidir una medida precisa y permiten adaptar la página perfectamente al navegador.

Se define un bloque div en el que irá el mapa, definiendo también unos estilos CSS.

Luego, se define el bloque asociado al mapa de leaflet. En él, se definen:

centerPoint: donde se ubica el mapa al inicio de cargar la página.

zoomLevel: la cantidad de zoom que hace el mapa al inicio de cargar la página.

markers: los marcadores que aparecen en el mapa (el array con los puntos de recogida)

class: reglas CSS para el mapa (en este caso, el tamaño que tendrá, el cuál será ocupar todo el espacio disponible del div)

Las propiedades **centerPoint**, **zoomLevel**, y **markers**, tienen el signo “:” delante. Esto se usa para indicar que son variables PHP y que Laravel las tiene que tratar como tal. En el caso de **class**, al no tener “:”, es tratado como un atributo HTML.

Archivo web.php

```
routes > web.php
You, hace 3 horas | 2 authors (You and one other)
1 <?php
2
3 use Illuminate\Support\Facades\Route;
4 use App\Http\Controllers\MapController;
5
6 // Route::get('/', function () {
7 //     return view('welcome');
8 // });
9
10 Route::get('/', [MapController::class, 'index'])->name('mapa');
```

Archivo web.php

Este es el archivo que enlaza los endpoints con cada función de los controladores. En este caso, cuando carga la página principal, se enlaza con la función index del MapController, que es la que contiene la lógica para devolver el mapa con los marcadores.

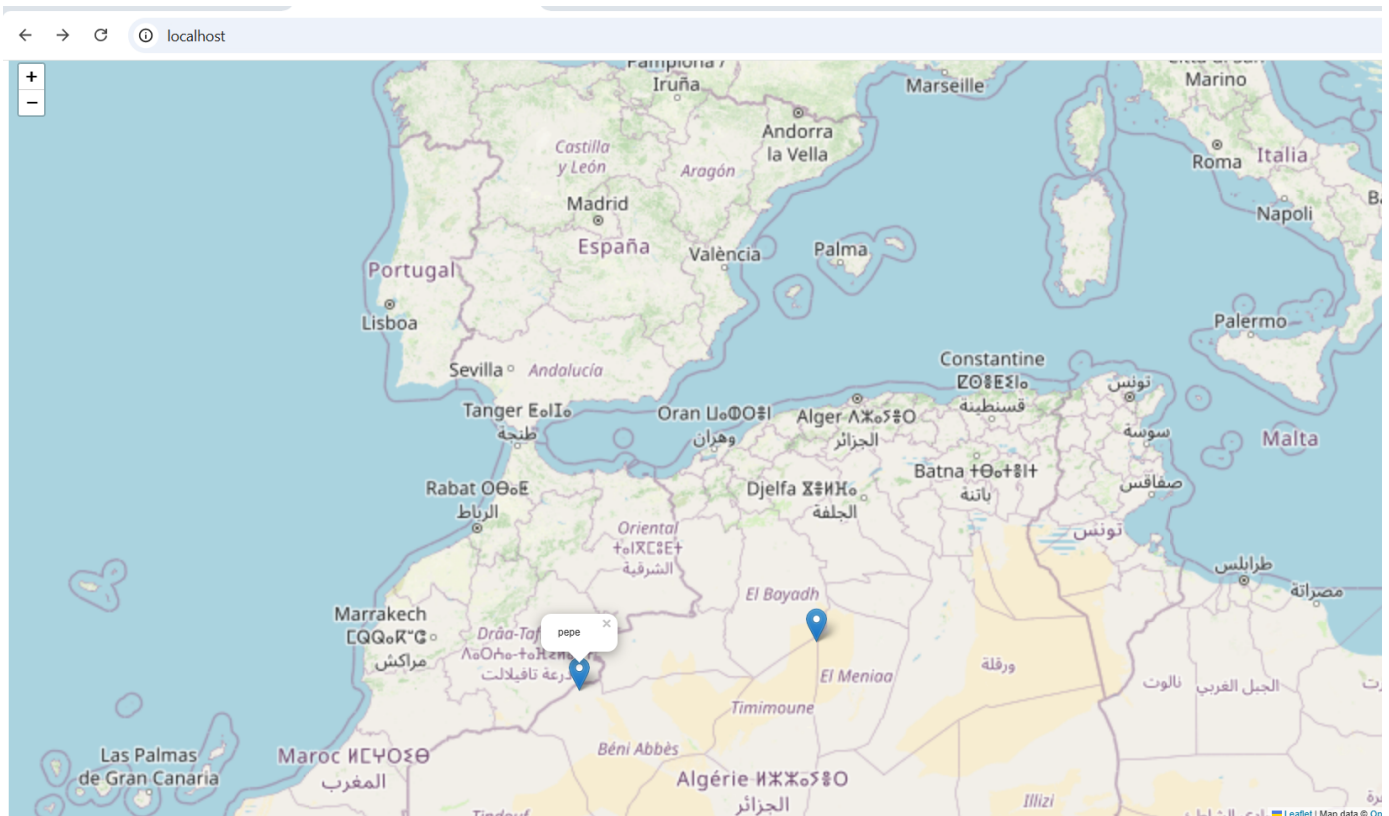
Resultado en el navegador

SELECT * FROM seller_profiles LIMIT 100							
Resultados de búsqueda							
	seller_id	store_name	description	nif	banner_url	created_at	updated_at
	bigint unsigned	varchar(255)	text	char(9)	varchar(255)	timestamp	timestamp
> 1		pepe	(NULL)	12345678L	(NULL)	(NULL)	(NULL)
> 2		pepa	(NULL)	98765432P	(NULL)	(NULL)	(NULL)

Datos de los vendedores en la base de datos

	Q	id bigint unsigned	seller_id bigint unsigned	latitude decimal(11,8)	longitude decimal(11,8)	address varchar(255)	created_at timestamp	updated_at timestamp
	>	1	1	30.42000000	-4.20000000	k	(NULL)	(NULL)
	>	2	2	31.50000000	2.00000000	p	(NULL)	(NULL)

Puntos de recogida de la base de datos



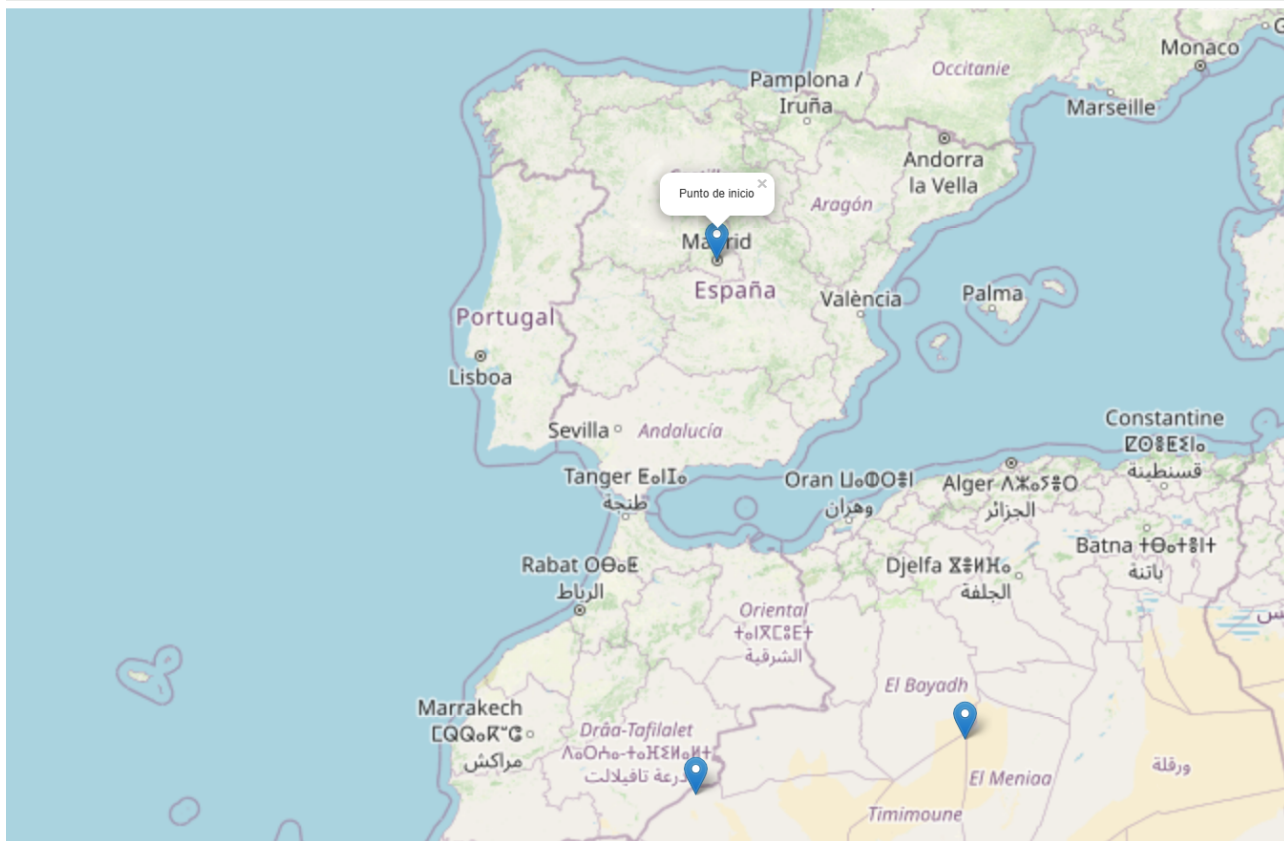
Mapa con los marcadores de los puntos de recogida (al hacer clic en cada marcador, sale el nombre de la empresa del respectivo vendedor)

Añadir un marcador de inicio

También es interesante hacer saber la ubicación de la que partes, no solo los puntos de recogida. Para indicarlo, hay que integrar un marcador más al array de los marcadores.

```
app > Http > Controllers > MapController.php
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\PickupPoint;
6  use Illuminate\Http\Request;
7
8  class MapController extends Controller
9  {
10     public function index()
11     {
12         $puntos = PickupPoint::with('sellerProfile')->get();
13
14         $marcadores = $puntos->map(function ($punto) {
15             return [
16                 'lat' => $punto->latitude,
17                 'long' => $punto->longitude,
18                 'info' => $punto->sellerProfile->store_name
19             ];
20         });
21
22         $latInicial = 40.416;
23         $longInicial = -3.703;
24
25         $marcadores->push([
26             'lat' => $latInicial,
27             'long' => $longInicial,
28             'info' => 'Punto de inicio'
29         ]);
30
31         return view('mapa', ['marcadores' => $marcadores->toArray()]);
32     }
33 }
34
```

Como los arrays no tienen funciones, hay que mantener la variable de **marcadores** como una colección para poder añadir el marcador inicial, y luego convertirlo a un array al pasarlo a la vista.



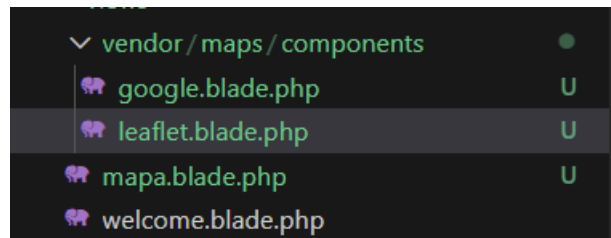
Mapa con marcador de inicio

Personalizar el mapa

Si quisiéramos añadir configuraciones personalizadas que las configuraciones básicas no permiten, hay que acceder a los archivos de configuración y editarlos para adaptarlos a lo que queremos.

Para acceder a dichos archivos, dentro de la terminal introducimos el comando **`./vendor/bin/sail artisan vendor:publish --provider="Larswiegers\LaravelMaps\LaravelMapsServiceProvider"`**

Luego de hacer esto, se podrán ver dos archivos:



Archivos de configuración

Hay un archivo de configuración de mapas Leaflet y otro de Google. He optado por hacer uso de leaflet para evitar problemas de pago con google maps.

```

resources > views > vendor > maps > components > leaflet.blade.php
1 <link rel="stylesheet" href="https://unpkg.com/leaflet@1.7.1/dist/leaflet.css"
2   integrity="sha512-xodZBNTC5n17Xt2atTPuE1HxjVMSvLVW9ocqQUKlScc5CXdbqCmblAshQMAS6/keqq/SMZM219scrR4PsZChSR7A=="
3   crossorigin="" />
4
5 <style>
6   #{{ $mapId }} {
7     @if(! isset($attributes['style']))
8       height: 100vh;
9     @else
10      {{ $attributes['style'] }}
11    @endif
12  }
13 </style>
14
15 <div id="{{ $mapId }}" @if(isset($attributes['class']))
16   class="{{ $attributes['class'] }}"
17 @endif
18 ></div>
19
20 Make sure you put this AFTER Leaflet's CSS
21 <script src="{{ 'https://unpkg.com/leaflet@' . $leafletVersion . '/dist/leaflet.js' }}"
22   crossorigin="" ></script>
23 <script>
24
25   var mymap = L.map('{{ $mapId }}').setView([{{ $centerPoint['lat'] ?? $centerPoint[0] }}, {{ $centerPoint['long'] ?? $centerPoint[1] }}, {{ $zoomLevel ?? 13 }});
26   @foreach($markers as $marker)
27     @if(isset($marker['icon']))
28       var icon = L.icon({
29         iconUrl: '{{ $marker['icon'] }}',
30         iconSize: [{{ $marker['iconSizeX'] ?? 32 }}, {{ $marker['iconSizeY'] ?? 32 }}],
31       });
32     @endif
33     var marker = L.marker([{{ $marker['lat'] ?? $marker[0] }}, {{ $marker['long'] ?? $marker[1] }}]
34     @if(isset($marker['icon']))
35       , {icon: icon}
36     @endif
37   );
38   mymap.addLayer(marker);
39 @endforeach
40 </script>

```

Archivo de configuración de leaflet

Este es el archivo de configuración que viene por defecto luego de ejecutar el comando. De forma resumida, el archivo tiene el enlace para importar los estilos del mapa, el div donde cargará el mapa, un script para importar todos los comandos de leaflet relacionado con los mapas y otro script con la configuración del mapa.

Nos vamos a centrar en esta línea de código:

```
var mymap = L.map('{{mapId}}').setView([{{centerPoint['lat'] ?? $centerPoint[0]}}, {{centerPoint['long'] ?? $centerPoint[1]}}, {{zoomLevel}}];
```

Código de creación del mapa

Esta línea de código es la que se encarga de crear un mapa en el div con el id asignado (**mapId**) y establecer la **vista** (el **centrado**, donde se posicionará geográficamente el mapa al cargar y el **zoom inicial** con el que enfoca el mapa).

En cuanto a la variable **centerPoint**, nos fijamos en que se le hace referencia de dos maneras. O bien se accede a la propiedad latitud mediante **\$centerPoint['lat']** o bien, como esta variable contiene un array, la latitud está en la posición 0 y la longitud en la posición 1, por lo que para acceder a la latitud en este caso, se hace con **\$centerPoint[0]**. Los signos de puntuación '??' se usan como un if. Si dentro de la variable centerPoint existe una propiedad llamada 'lat', entonces accede a su valor, si no, accede al valor de la posición 0 del array.

Ahora, en caso de que queramos personalizar el mapa (por ejemplo, lo que he querido hacer era limitar el mapa para que no pudieras arrastrar más allá de España y también no poder sobrepasar unos ciertos niveles de zoom, para no poder ver el mapamundi entero), existe un objeto llamado **options**, que como el nombre indica, contiene todas las personalizaciones que le añadas para ajustar el mapa a tus necesidades.

```
23 var mymap = L.map('{{mapId}}', {
24     minZoom: 7,
25     maxZoom: 18,
26     maxBounds: [
27         [25.0, -20.0],
28         [55.0, 15.0]
29     ],
30     maxBoundsViscosity: 1.0
31 }).setView([{{centerPoint['lat'] ?? $centerPoint[0]}}, {{centerPoint['long'] ?? $centerPoint[1]}}, {{zoomLevel}}];
32
```

Personalización del mapa

Ahora, voy a detallar qué hace cada propiedad:

minZoom: hace referencia al zoom mínimo que se puede hacer. De manera más sencilla, el zoom permite valores del 0 al 18, siendo 0 el zoom más pequeño del mundo (permite ver el mapamundi) y 18 el zoom más grande (permitiendo ver las calles de una ciudad al detalle). Entonces con el minZoom, podemos restringir el zoom para que no se pueda ir más allá de ver un país (que es el ajuste que he aplicado). De esta manera, no se puede ver el mapamundi por mucho que intentes minimizar el mapa.

maxZoom: lo contrario al minZoom, es el zoom máximo que se puede hacer. Como el valor más grande es 18, permitiendo ver las calles al detalle, puedes poner un valor como 15, para así, que el usuario no pueda maximizar el mapa más allá de este valor. Por ahora, lo he dejado en 18, pero si más adelante no nos parece bien este ajuste, siempre se puede cambiar.

maxBounds: restringe hasta cuánto el usuario puede arrastrar el mapa. Un ejemplo claro es este código, dónde he limitado el movimiento del mapa para que no puedas salir de España (falta ajustarlo un poco más) y así no puedas irte al continente americano. Como esta aplicación está hecha para Tavernes, no tiene sentido permitir al usuario irse a otro país o continente.

Sobre cómo funciona esta propiedad, como ya he dicho, restringe el movimiento, y lo hace en forma de cuadrado. De manera que, se le pueden pasar dos objetos o dos arrays. Uno de ellos hace referencia a la esquina inferior izquierda, y el otro a la esquina superior derecha.

Usage example

```
var p1 = L.point(10, 10),  
    p2 = L.point(40, 60),  
    bounds = L.bounds(p1, p2);
```

All Leaflet methods that accept [Bounds](#) objects also accept them in a simple Array form (unless noted otherwise), so the bounds example above can be passed like this:

```
otherBounds.intersects([[10, 10], [40, 60]]);
```

Ejemplo de uso de la web oficial

maxBoundsViscosity: esta propiedad acompaña a la propiedad maxBounds, y es la que permite que el usuario no se salga de los límites establecidos. Acepta valores de entre 0.0 y 1.0, de manera que si no se pone esta propiedad es como si tuviera valor 0.0 . Su funcionamiento es el siguiente: cuando tiene un valor pequeño (pongamos 0.0) lo que hace es permitir al usuario salirse un poco de los límites establecidos por maxBounds, pero al momento en que se suelte el ratón, devolverá al usuario al rango permitido por maxBounds. Cuanto más alto sea el valor, más difícil será salirse de los límites, de manera que si se deja en 1.0, el usuario no podrá salirse ni un milímetro de los límites establecidos.

(Quería adjuntar un vídeo para mostrar los resultados, pero no me ha sido posible)